

STAT 427

Illinois Campus Cluster Usage Analysis

Rushil Bhupta
Jiachen Cong
Vladislav Fedorov
Rishab Sakalkale
Yanlin Wang



May 2026

Abstract

This report presents a data-driven analysis of job scheduling behavior and resource utilization on the Illinois Campus Cluster, a shared high-performance computing system managed through the SLURM workload manager. Using a full year of job-level accounting logs covering over six million job submissions, we trace how inefficiency emerges from the interaction of workload structure, user behavior, and scheduler response.

The analysis proceeds through four stages. First, we establish a system baseline revealing that both waiting time and runtime are heavily right-skewed: most jobs start quickly and complete in minutes, but a small subset experiences extreme delays and consumes disproportionate resources. Non-completed jobs account for approximately 25% of all submissions. Second, we quantify sources of heterogeneity in workload and user behavior. Through k-means clustering on resource request features, we identify three distinct workload types - Light, Medium, and Heavy - with Heavy jobs exhibiting GPU-intensive profiles and the strongest tail behavior in waiting time. At the user level, the top 10% of users account for 93.66% of job submissions and nearly 100% of GPU consumption, with Gini coefficients of 0.945 and 0.998 respectively, confirming near-monopoly resource concentration.

Third, we examine how this heterogeneity propagates into resource contention and queue congestion. A Mann-Whitney U test confirms that GPU jobs experience significantly longer waiting times than CPU-only jobs ($U = 1.05 \times 10^{12}$, $p \approx 0$). Heavy jobs exhibit the shortest median waiting time yet the largest mean, reflecting tail-driven contention from long-running jobs that lock scarce resources. Queue congestion is temporally concentrated: median wait times are stable across submission hours, while mean wait times spike episodically, driven by burst-period overload. The Spearman correlation between runtime and waiting time is only 0.4238, confirming that delays cannot be attributed to job length alone.

Fourth, we characterize system-level inefficiency in resource allocation. The median time utilization is 0.57%, with jobs running approximately 200 times shorter than their reserved walltime, corresponding to roughly 800 million unused CPU-hours. Memory efficiency is similarly poor, with a median of 1.48% and 91.5% of jobs using less than half their allocated memory. To capture multidimensional inefficiency, we introduce an eight-category job efficiency metric based on binary classification of memory, CPU, and time efficiency. A chi-squared test ($\chi^2 = 55,557$, $p \approx 0$) with Bonferroni-corrected pairwise comparisons establishes that memory efficiency is the critical fault boundary: groups with low memory efficiency exhibit failure rates of 10–26%, while all high-memory-efficiency groups cluster near 2.5% and are statistically indistinguishable from one another.

Finally, predictive models for job waiting time are developed separately for GPU and CPU-only jobs using Random Forest regression on log-transformed response variables, achieving RMSLE of 0.51 and 0.66 respectively. A companion classifier identifies heavy users with 85% accuracy and ROC-AUC of 0.82. Together, these models support proactive scheduling decisions and targeted intervention.

Taken together, these findings establish that the Illinois Campus Cluster operates efficiently in its central tendency but is stressed by tail behavior. Inefficiency is not randomly distributed: it is structurally concentrated among a small fraction of users, a specific workload type, and jobs that underutilize memory. Resource waste is both large in scale and predictable from information available at submission time. The interaction of workload heterogeneity, user concentration, and temporal burst patterns jointly determines which jobs experience extreme delays, and no single factor in isolation is sufficient to explain observed outcomes. Based on these conclusions, we recommend the adoption of fairshare scheduling policies and GPU quota enforcement to reduce resource monopolization; deployment of personalized per-user efficiency summaries to encourage more accurate resource requests; targeted intervention for jobs falling in low-memory efficiency groups, which account for the majority of cluster failures; and integration of the developed predictive models into scheduling workflows to enable proactive identification of delay-prone and resource-intensive submissions before they enter the queue.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Framework and Core Insight	4
2	Problem Statement and Objectives	5
2.1	Problem Definition	5
2.2	Project Objectives	5
2.3	Datasets Preprocessing	6
3	System Baseline: Overall Cluster Behavior	7
3.1	Submission patterns (time-of-day / weekday).	7
3.2	Waiting time distribution and runtime distribution.	7
3.3	Resource requests (CPU / GPU / memory).	8
3.4	Job state distribution.	9
3.5	Terminal state structure.	10
3.6	Time-series of cluster load and failures.	11
3.7	System Behavior EDA Summary	11
4	Sources of Heterogeneity (What + Who)	12
4.1	Workload Heterogeneity	12
4.2	User Behavior and Resource Inequality	13
4.3	Sectional Summary	17
5	Contention and Congestion (How)	17
5.1	Resource Contention	17
5.1.1	GPU Jobs Experience Longer Waiting Times	17
5.1.2	Workload-Level Waiting Time Heterogeneity	18
5.1.3	Workload-Level Runtime Differences	19
5.2	Queue Congestion	20
5.2.1	Temporal Variation in Waiting Time	20
5.2.2	Relationship Between Runtime and Waiting Time	20
5.3	Contention and Congestion Sectional Summary	21
6	System Inefficiency	22
6.1	System Inefficiency in Cluster Resource Requests	22
6.2	Inefficiency Analysis Beyond Runtime	22
6.2.1	Memory Efficiency	22
6.3	Potential Improvements	22
6.3.1	User Feedback and Personalized Usage Summaries	23
6.4	Job Category Efficiency Metric	24
6.4.1	Group Distribution and Job Outcomes	25
6.4.2	Statistical Validation: Chi-Squared Test of Independence	25
6.4.3	Within-Group Memory Efficiency Distributions	26
6.5	Sectional Summary	26
7	Predictive Modeling	27
7.1	Predicting GPU Job Waiting Time	27
7.2	Predicting CPU-only Job Waiting Time	28
7.3	Predicting Heavy Users and System-Level Behavior	30
7.4	Sectional Summary	30

8 Conclusion and Recommendations	31
8.1 Key Findings and System Implications	31
8.2 Recommendations to the Client	31
8.3 Limitations & Future Work	32
Appendix A: Code and File Structure	33

1 Introduction

1.1 Background and Motivation

High-performance computing (HPC) systems underpin modern scientific discovery, engineering simulation, and, increasingly, the training and evaluation of large-scale machine learning models. A typical production cluster aggregates thousands of compute nodes shared across diverse user communities, each submitting jobs with heterogeneous resource demands, runtime profiles, and priority levels. In this multi-tenant environment, the scheduler plays a central role: it allocates finite compute resources among competing requests, and even modest improvements in efficiency translate into meaningful gains in throughput, energy use, and scientific productivity.

The Illinois Campus Cluster is a representative example of such a shared HPC system. It provides campus-wide computing resources to researchers across a wide range of disciplines, with jobs submitted and managed through the SLURM workload manager. During operation, the system automatically records detailed job-level logs, including submission time, requested resources, queue information, and realized execution outcomes.



Figure 1: Overview of the Illinois Campus Cluster system interface.

Despite substantial investment in scheduling algorithms and hardware, operational HPC systems continue to exhibit heterogeneous **efficiency patterns** in practice. Wait times span several orders of magnitude across jobs submitted to the same queue, with most jobs starting quickly while a subset experiences substantial delays; a small fraction of users account for the bulk of demand; and a large share of requested runtime is never actually used. As we will show, the Illinois Campus Cluster logs themselves suggest that much of this variability does not arise from scheduler design alone, but from the interaction between workload structure and user behavior. Classical policies such as first-come-first-served with backfilling and priority aging are inherently reactive: they respond to congestion after it has already formed rather than anticipating it from the characteristics of incoming jobs and the users who submit them.

The availability of a full year of SLURM accounting logs enables a complementary, data-driven perspective. Rather than treating workload and user behavior as fixed inputs, we can explicitly trace their role in shaping system dynamics, and ask where along that chain the largest efficiency losses actually occur.

As we will show, these inefficiencies are not uniformly distributed across the system, but are concentrated in a subset of jobs that experience extreme delays.

1.2 Framework and Core Insight

To organize the analysis, we adopt a simple four-stage framework describing how inefficiency emerges in a shared HPC system:

This framework reflects three observations from the SLURM job traces. First, **jobs** are not exchangeable: their resource requirements vary widely in CPU, memory, and GPU usage, and this heterogeneity is a primary driver of contention. Second, **users** are not exchangeable: submission frequency, workload profiles, and temporal patterns differ substantially across users and research groups, so user identity carries information

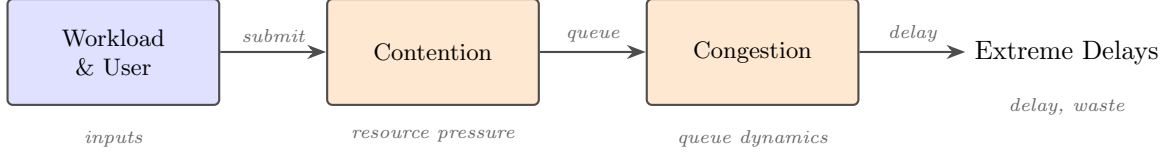


Figure 2: From workload and user characteristics to tail-driven delays.

beyond job-level features alone. Third, **contention manifests** as queue congestion, which in turn produces inefficiency in the form of long waiting times, idle resources, and cascading delays for subsequent jobs.

The framework also gives the report its structure. Each stage of the pipeline is examined in turn: we first establish a system baseline of overall cluster behavior, then quantify the heterogeneity in workload and user inputs that feeds the pipeline, and then trace how that heterogeneity propagates into contention, congestion, and ultimately system-level inefficiency. The final stage synthesizes these findings into a predictive model that flags inefficiency-prone submissions at submission time, and into a set of concrete recommendations for the Illinois Campus Cluster operations team.

2 Problem Statement and Objectives

2.1 Problem Definition

The Illinois Campus Cluster, like most shared HPC systems, exhibits substantial variability between its nominal capacity and the efficiency experienced by different jobs. Most jobs start quickly, while a non-negligible fraction experiences substantially longer delays; some users submit a handful of carefully sized requests while others dominate entire partitions; large fractions of requested runtime go unused. These symptoms are visible in the accounting logs but **not directly explained by them**, because the underlying mechanism is an interaction between *who* submits jobs, *what* they submit, and *how* the scheduler responds under load.

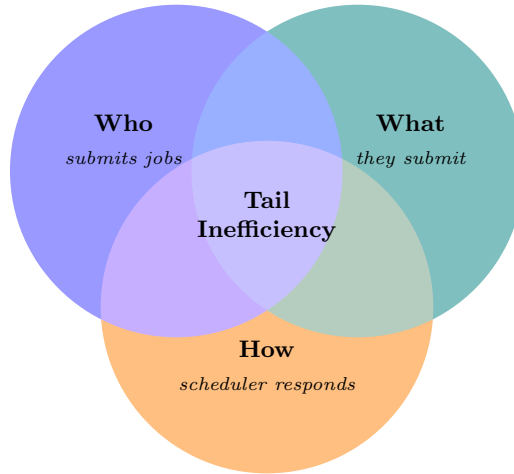


Figure 3: Efficiency losses on the cluster arise from 3 dimensions

2.2 Project Objectives

The project is organized around four objectives that follow the pipeline end to end:

1. **Establish a system baseline.** Characterize overall cluster behavior in 2025 including submission patterns, waiting and runtime distributions, resource requests, and job outcomes to provide a shared reference point for all subsequent analysis.

2. **Quantify sources of heterogeneity in workload and user behavior.** Identify distinct workload types through clustering, and measure the concentration of demand across users and departments, including inequality in CPU and GPU usage.
3. **Trace how heterogeneity propagates into contention, congestion, and localized inefficiency.** Link dominant users and heavy workloads to observed queue pressure; examine how waiting times scale with workload and runtime; and quantify the gap between requested and actual resource use, including unused CPU-hours at the system level.
4. **Translate findings into predictive and operational recommendations.** Build a two-stage predictive model that flags inefficiency-prone submissions at submission time, and synthesize the analysis into concrete recommendations for the Illinois Campus Cluster operations team, covering fair-share, quotas, runtime feedback, and related policy levers.

2.3 Datasets Preprocessing

Data Source Selection

The raw data are organized into multiple folders corresponding to different pipelines and use cases. Since the project focuses on cluster-wide job submission behavior and execution schedules, we use the production accounting logs under `whole-cluster-usage/`, which represent the full population of jobs.

We do not use the `statclass/` dataset, as it is a small instructional subset and not representative of overall cluster usage. We also exclude auxiliary outputs (e.g., `2025_seff/`), which contain derived efficiency summaries rather than primary scheduling records.

NAME	UPDATED ↓	SIZE
statclass	Feb 5, 2026 by Wyatt Ma...	12 Files
whole-cluster-usage	Feb 5, 2026 by Jedidiah ...	24 Files
slurm_all_jobs_jan15_oct2_2025.zip	Feb 2, 2026 by cheller	184 MB

Figure 4: Overview of the data directory structure.

Raw Data and Preprocessing

The 2025 accounting data consist of twelve monthly log files with a consistent schema, which we concatenate into a single dataset covering the full year. Preprocessing includes:

- Normalizing missing values (e.g., `unknown`, `none`, `n/a`)
- Restricting to jobs submitted in 2025
- Simplifying job state labels

We then construct the key variables used downstream:

- **Requested memory (MB):** converted from `ReqMem`
- **Waiting time:** `Start - Submit`
- **Runtime:** `End - Start`

Job-Level Dataset Construction

The merged logs may contain multiple records for a single job due to different lifecycle stages (e.g., pending, running, completed). To avoid double-counting, we construct job-level datasets with exactly one row per JobIDRaw.

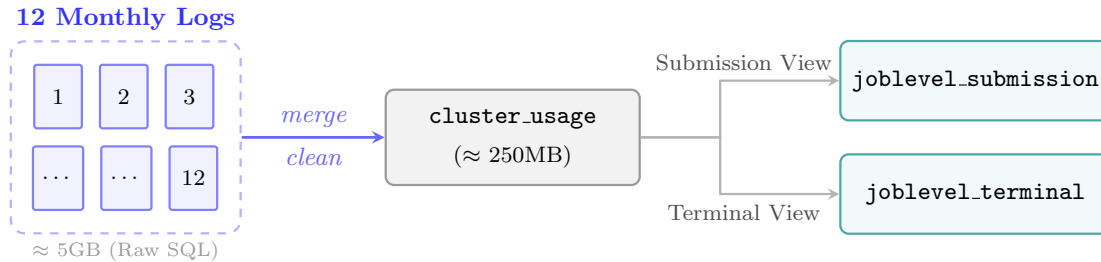


Figure 5: Data construction pipeline from raw logs to job-level datasets.

As shown in Figure 5, the monthly logs are first merged and cleaned into a unified dataset. We then construct two job-level views:

- **Submission view:** earliest record per job, used for analyzing submission behavior (6,025,326 rows $\times$ 16 columns).
- **Terminal view:** final or most complete record per job, used for analyzing execution outcomes (6,025,326 rows $\times$ 24 columns).

Final Datasets and Analytical Approach

The analysis is based on three datasets:

- `master_2025_raw_valid.csv.gz` — cleaned row-level data
- `master_2025_joblevel_submission.csv.gz` — job-level submission view
- `master_2025_joblevel_terminal.csv.gz` — job-level execution view

The two job-level datasets are used in all downstream analyses to ensure consistent interpretation at the job level. Methodologically, the project combines descriptive analysis to trace evidence at each stage of the pipeline, with predictive modeling in the final stage to synthesize these findings into a forward-looking tool that flags inefficiency-prone submissions before they enter the queue.

3 System Baseline: Overall Cluster Behavior

3.1 Submission patterns (time-of-day / weekday).

As shown in Figure 6, submissions are relatively consistent across weekdays, suggesting stable usage throughout the week. There is a small peak of submission at the start of a week. However, job submissions show a clear hourly pattern, with low activity during early morning hours and peak submissions in the afternoon and late evening. There is a slight drop in submissions during the early evening period (7 to 8 P.M.). The hourly pattern is consistent with students’ daily activity schedules.

3.2 Waiting time distribution and runtime distribution.

Figure 7 shows that waiting time is highly skewed, with most jobs starting quickly (the median of waiting time is 0.42 hours), but a subset experiencing long delays, about 10% of jobs have waiting time more than 13 hours.

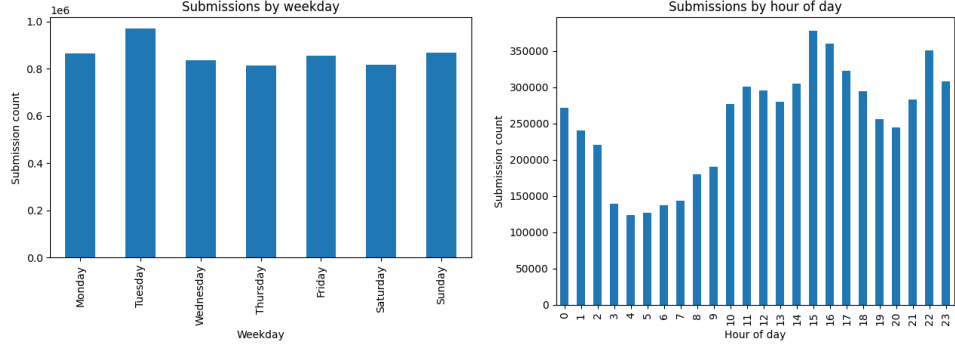


Figure 6: Job submissions by weekday and hour of day.

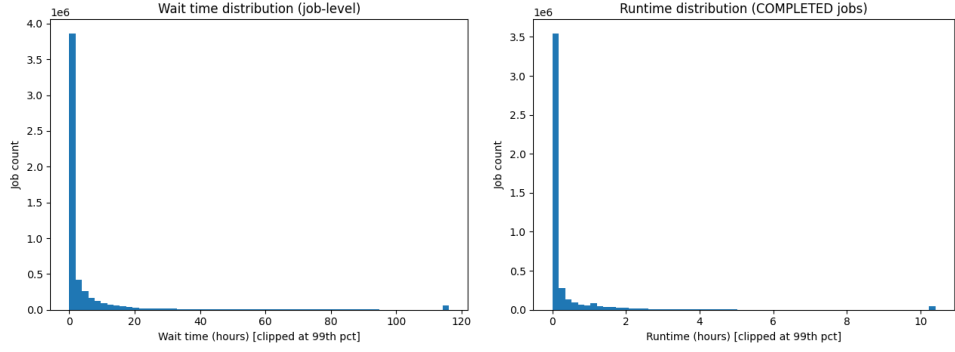


Figure 7: Distribution of waiting time and runtime.

Runtime exhibits a similar right-skewed pattern, where most jobs complete quickly but small amount of jobs run significantly longer. The median of runtime is 0.02 hours; 1% of jobs have runtime more than 10 hours.

3.3 Resource requests (CPU / GPU / memory).

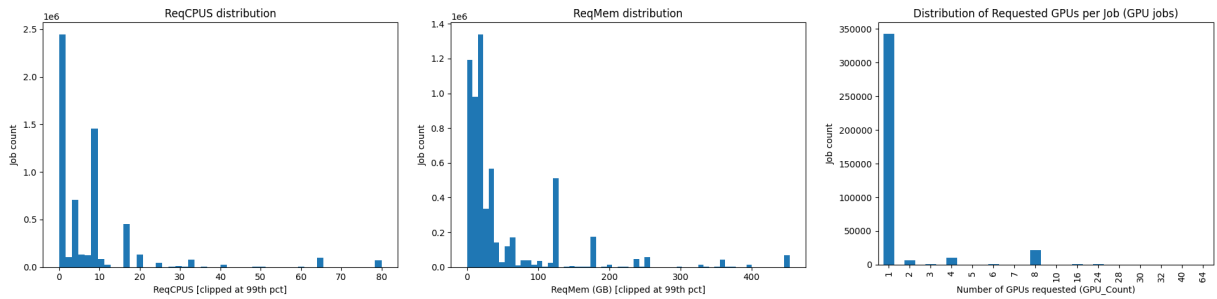


Figure 8: Distribution of requested CPU, memory, and GPU resources.

As shown in Figure 8, most jobs request relatively small amounts of CPU and memory resources. The median CPU request is 4 CPUs, and the 90th percentile is 16 CPUs. For memory, the median request is about 16 GB, and the 90th percentile is approximately 125 GB.

For GPU jobs, about 89% of jobs request only one GPU, while only around 6% of jobs request five or more GPUs. Although most GPU jobs are small, there are a few very large GPU requests, such as jobs requesting 40 or 64 GPUs.

3.4 Job state distribution.

Below is the counts and rates of each job state, and cluster-wide counts of these states over time:

	job_count	job_pct	row_count	row_pct
outcome				
Cancelled	639041	10.59	641112	10.50
Completed	4644274	76.98	4662342	76.36
Failed	585157	9.70	586554	9.61
Node Failed	218	0.00	220	0.00
Other	845	0.01	850	0.01
Out of Memory	43313	0.72	43546	0.71
Requeued	9	0.00	9	0.00
Running	301	0.00	3600	0.06
Stuck PENDING	3791	0.06	45464	0.74
Timeout	116144	1.93	122413	2.00
Total unique jobs: 6033093				
Total rows: 6106110				

Figure 9: Job State Counts

We can note that:

- 76% of jobs are completed
- 43k jobs are out-of-memory
- 20% of jobs are either cancelled or failed

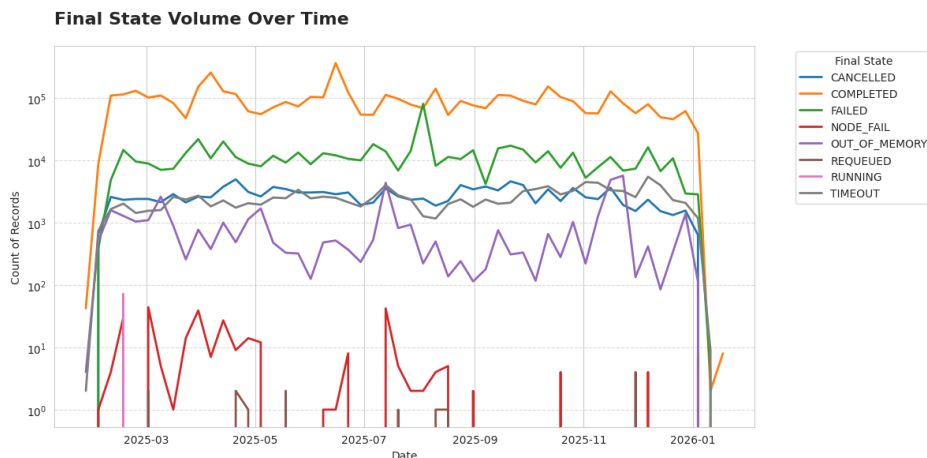


Figure 10: Job State Counts Over Time

Note that in the original data, there were several thousand unique job states following the format "CANCELLED by #####", with the number representing the Job ID of a different job. While these job states are simply represented as CANCELLED in the plots above, additional analysis into an example of such a state was performed to understand the impact of these jobs - are they one-off jobs run by a user to quickly kill off jobs or part of an unusual/catastrophic/unprovoked event?

One of the largest-impact examples is the job with ID 1147665. This job killed 21586 jobs with names like:

- ddp-h200-torch
- ddp-l40s-torch
- aims_light_model_test
- full_ae-h200

Importantly, all these jobs killed by job 1147665 were created by one user. Similar analysis on other, large-impact job-killers reveals a similar dynamic: often times, users submit jobs that kill a large amount of their own jobs. Because these are intentional, controlled job cancellations, we are justified in labeling all "CANCELLED by #####" jobs as "CANCELLED" jobs. Further confirmation from our meeting with our client supports this conclusion.

3.5 Terminal state structure.

Now, as mentioned in the introduction, one of the initial motives in analyzing job states was to track a job's state over time. Below is a list of the average number of rows per unique job organized by their final job state.

	count	mean	std	min	25%	50%	75%	max
outcome								
Cancelled	639041.0	1.003241	0.065675	1.0	1.0	1.0	1.0	10.0
Completed	4644274.0	1.003890	0.062251	1.0	1.0	1.0	1.0	2.0
Failed	585157.0	1.002387	0.048943	1.0	1.0	1.0	1.0	3.0
Node Failed	218.0	1.009174	0.095562	1.0	1.0	1.0	1.0	2.0
Other	845.0	1.005917	0.076741	1.0	1.0	1.0	1.0	2.0
Out of Memory	43313.0	1.005379	0.073148	1.0	1.0	1.0	1.0	2.0
Requeued	9.0	1.000000	0.000000	1.0	1.0	1.0	1.0	1.0
Running	301.0	11.960133	0.267841	10.0	12.0	12.0	12.0	12.0
Stuck PENDING	3791.0	11.992614	0.112296	7.0	12.0	12.0	12.0	12.0
Timeout	116144.0	1.053976	0.237671	1.0	1.0	1.0	1.0	3.0

Figure 11: Job State Counts

We notice that:

- 1 unique job = 1 row in the SLURM dataset
- Only pending and running states have multiple rows per job

This unfortunately indicates that job states are terminal: we are unable, based on the current data, able to track the evolution of jobs.

This also indicates to us that the PENDING job state is even more uncommon than previously perceived, since most jobs get stuck in PENDING, and artificially add rows into the SLURM data - something that doesn't happen in other job states. As further confirmation, we notice that:

- 88.7% of PENDING jobs only request 1 CPU
- 70% of jobs requested by 1 user in the "astro-physics" group

Since the PENDING jobs come from a single user, we can remove them from future analyses.

3.6 Time-series of cluster load and failures.

Now, finally, we look at cluster-wide job states over time. Below is the plot of requested resources and incomplete jobs over time:

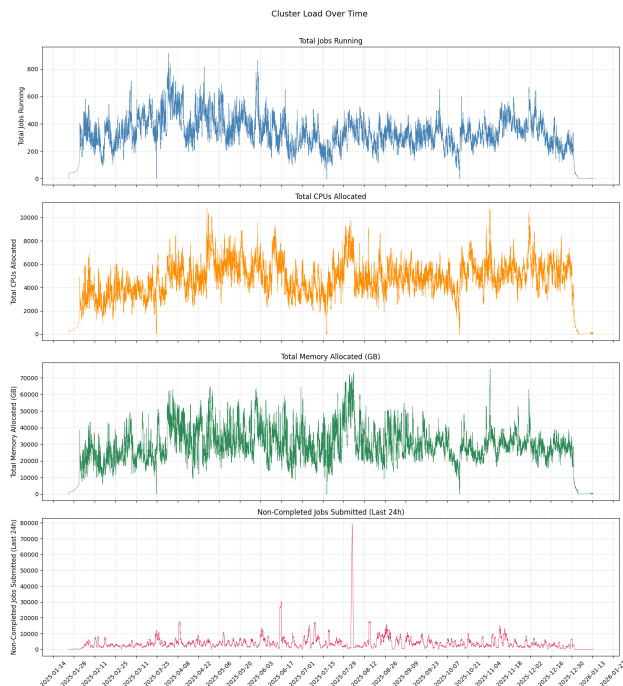


Figure 12: Job State Counts

From the plots above, we can see a major spike in incomplete events in late July, which seems to coincide with a sizable increase in requested resources. This motivates our later investigation into the correlation between job efficiency and job failures - since the plots above show that allocation spikes correspond to incomplete job spikes, perhaps their job inefficiencies are a driving factor in both high allocation and high failure rates.

3.7 System Behavior EDA Summary

Key Findings:

1. Job submissions are temporally structured: stable across weekdays, but vary by hour.
2. Both waiting time and runtime exhibit **heavy-tailed distributions**.
3. Most jobs request **small resources**, but extreme requests exist.
4. Non-complete jobs account for $\tilde{25}\%$ of all jobs!
5. Individual users create spikes of failing jobs, which put a strain on cluster resources - better education on careful review before submitting jobs might be necessary.

- **Takeaway:** The system is **not homogeneous**, variability is fundamental.
- **Next:** We investigate the **sources of this heterogeneity** in the following section.

4 Sources of Heterogeneity (What + Who)

4.1 Workload Heterogeneity

Workload in HPC systems is inherently multi-dimensional and cannot be adequately described by a single variable. Job resource requirements depend on multiple factors, including CPU, memory, and GPU usage, which jointly determine the overall computational intensity.

To address this, we construct a derived workload variable by clustering resource request features, allowing us to capture workload intensity in a data-driven manner. Specifically, we consider three key features: requested CPUs, GPU count, and memory (MB). Since these variables exhibit **strong right-skewness**, we apply a $\log(1+x)$ transformation to stabilize the scale, followed by standardization.

As shown in Figure 13, the log transformation reveals clearer structured patterns in the feature space, suggesting that resource requests are not continuously distributed but concentrated around a limited set of standard configurations.

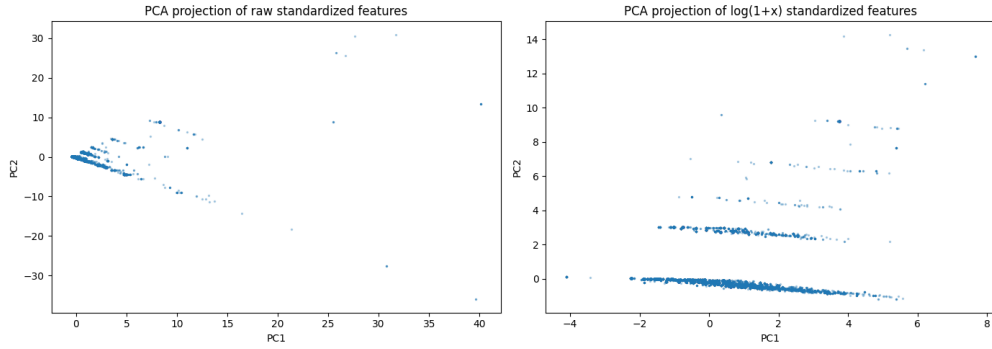


Figure 13: PCA projections before and after log transformation.

We then apply K-means clustering to the transformed feature space and determine the number of clusters using the elbow method. As illustrated in Figure 14, the inertia decreases rapidly from $k = 2$ to $k = 3$, after which the marginal improvement diminishes. Based on this analysis, we select $k = 3$ as a balance between model complexity and interpretability.

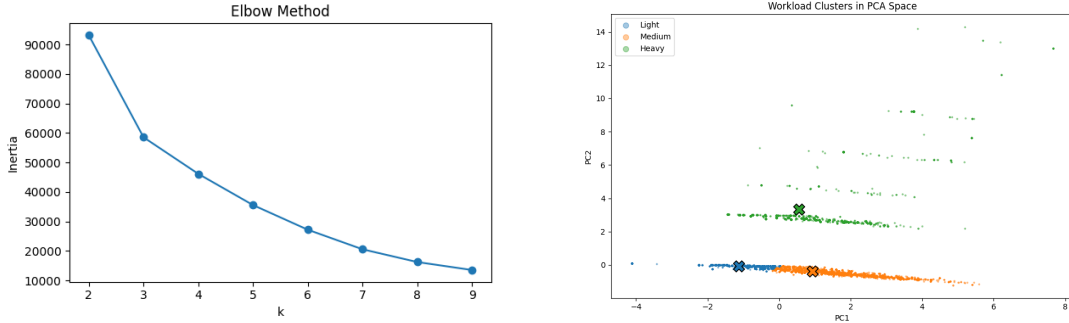


Figure 14: Left: Elbow method for selecting the number of clusters. Right: K-means clustering structure.

After clustering, we assign semantic labels to the clusters by **ranking their cluster centers according to average standardized resource intensity**. The lowest-intensity cluster is labeled *Light*, the middle cluster is labeled *Medium*, and the highest-intensity cluster is labeled *Heavy*. Thus, these workload categories are data-driven labels rather than manually specified thresholds. The resulting clusters correspond to distinct workload types:

1. **Light workload:** low CPU and memory usage, typically no GPU
2. **Medium workload:** CPU- and memory-intensive jobs

3. Heavy workload: GPU-intensive and high-memory jobs

To understand how these workload types are distributed across the system, we examine their empirical frequency. As shown in Figure 15, most jobs fall into light and medium workloads, while heavy workloads form a smaller but distinct group. This distribution indicates that although most jobs are relatively lightweight, a non-negligible fraction of resource-intensive jobs exists, which may play a disproportionate role in system-level behavior such as contention and congestion.

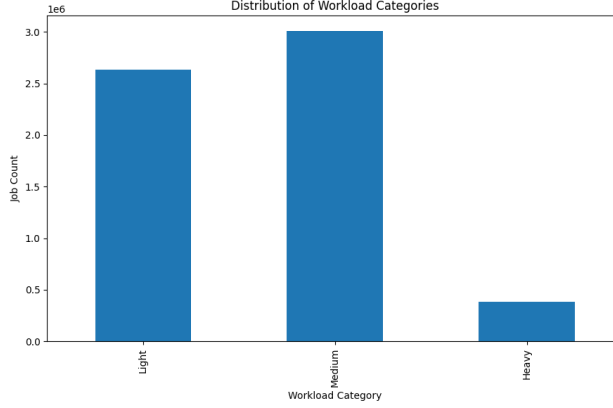


Figure 15: Distribution of workload categories.

These workload categories capture systematic differences in resource usage patterns and serve as a key variable in subsequent analyses.

4.2 User Behavior and Resource Inequality

This section examines how users and departments utilize cluster resources, with a focus on identifying imbalance, inefficiencies, and potential fairness issues in resource allocation.

- **Heavy user concentration (top 10% = 93.66% of submissions):**

There is a clear case of skewness in the distribution of jobs submitted by users. The analysis indicates that the top 10 percentage of users submit approximately 93.66 percentage of all jobs, suggesting high concentration of the workload on a few users. This form of skewness implies that most of the cluster workload is done by few powerful users while many users submit relatively few jobs.

Since most of the job submissions to the cluster are made by the top 10 percentage of the users, this will have important implications for overall cluster operation and fairness in the process. Due to their high contribution to the cluster, the scheduler will have to deal with a high load due to their repeated submissions of jobs.

Fair share scheduling algorithms need to be put in place in order to cater to the imbalance in such a way that users' priorities can be adjusted according to previous utilization levels. Job submission quotas during the most intensive times and priority decay for dominant users are recommended.

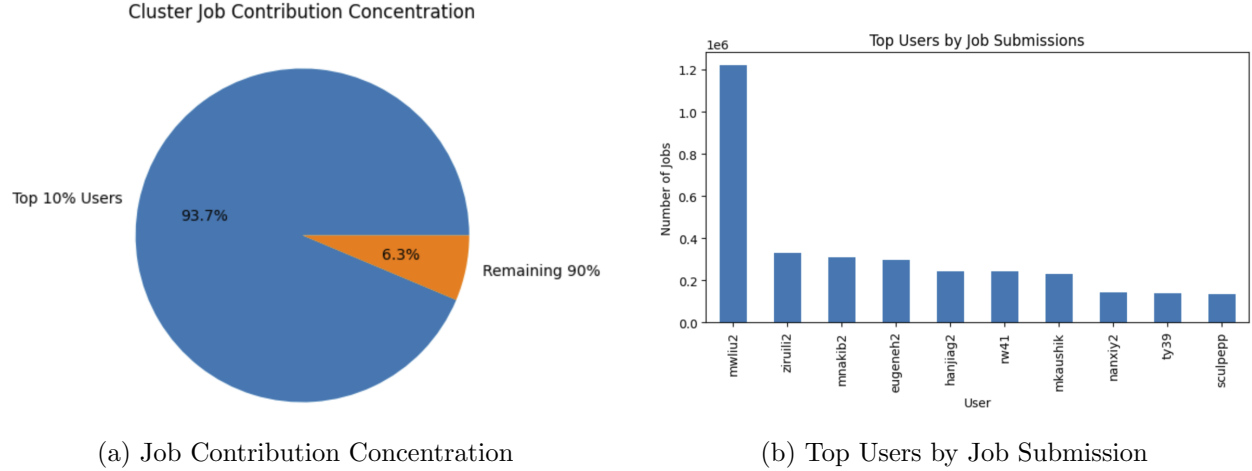


Figure 16: Heavy User Concentration and Top Users by Job Submission

- **GPU Resource Concentration (top 10% $\approx$ 100% of GPU demand):**

GPU usage is much more skewed towards fewer users than CPU usage. The top 10 percentage of GPU users use close to 100 percentage of all the GPUs used, which shows an extreme disparity in resource allocation to accelerators. It is clear from this point that the heavy usage of GPUs comes from a very small proportion of users.

The fact that such a large proportion of GPU usage is confined to a few users results in a huge disparity in resource access. It hinders other users from using GPUs, causing delays in job execution and ineffective use of costly computing resources, especially in machine learning and scientific applications.

Allocating GPU quotas to users or research groups and capping the use by each user can be an effective way of ensuring fairness. Another way would be to have multiple GPU queues based on workload sizes and allowing reallocation of idle GPU resources.

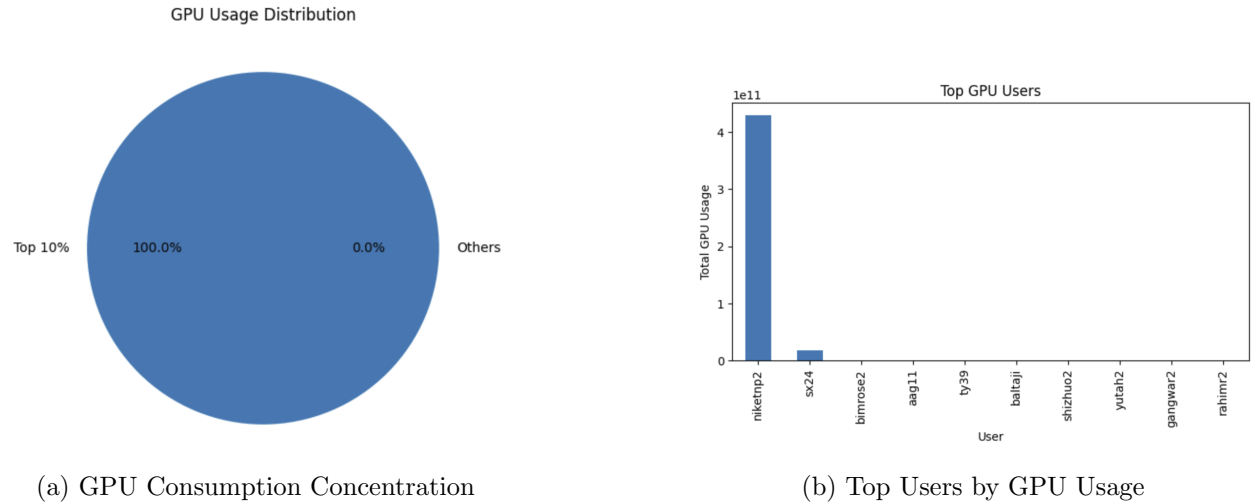


Figure 17: GPU Usage Monopoly

- **Department-level concentration (top 5 departments = 82.98% CPU):**

At the level of departmental analysis, it emerges that the top 5 departments account for about 82.98 percentage of total CPU use. It means that the use of clusters is highly concentrated in some departments. It might show unevenness in resource access and computation needs.

High concentration of the CPU use in just a few departments shows the inequality of institution-wide resource use. It can be a barrier for small or newly established research groups to gain access to sufficient computing power.

It will be essential to introduce departmental fair-share allocation policy that would balance out resource use. Moreover, unallocated shares can be redistributed to optimize their use.

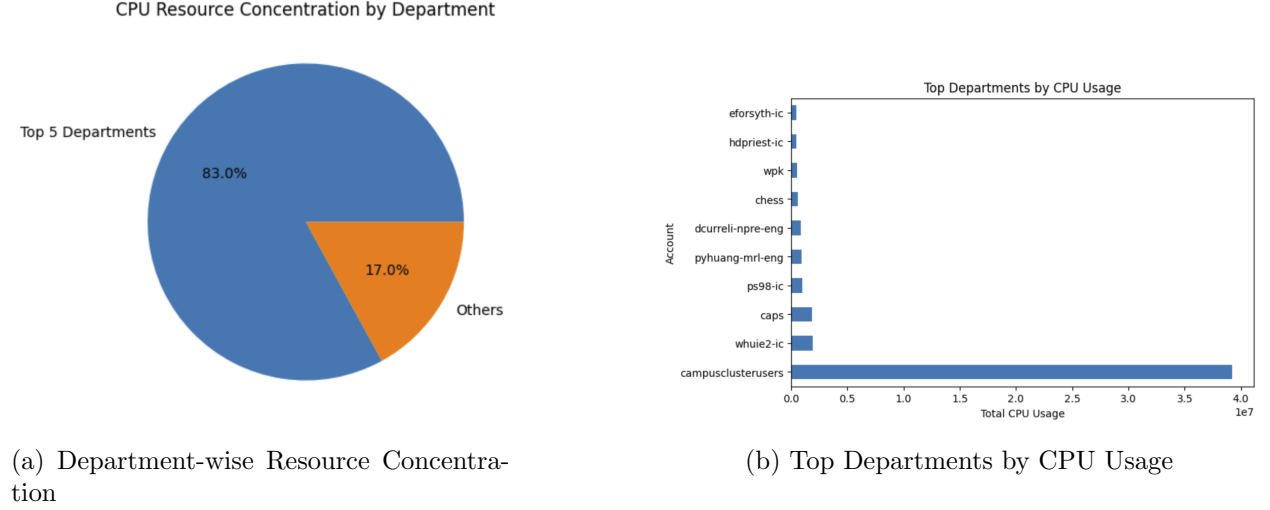


Figure 18: Top 5 Departments by CPU Usage

- **Job size distribution (45.15% tiny jobs ≤ 2 CPUs; 4.99% large ≥ 32 CPUs)**

The workload is highly polarized in terms of job size:

1. 45.15 percentage of jobs are tiny jobs (2 CPUs)
2. 4.99 percentage of jobs are large jobs (32 CPUs)

This indicates that most jobs are lightweight, while a small percentage of jobs are highly resource-intensive.

The inclusion of a larger number of very small jobs will increase the scheduling overhead since the scheduler will have to handle many small jobs. On the other hand, the larger jobs which are relatively few consume a lot of resources and block the nodes for a long period of time. The combination of these two creates inefficiencies in the whole process of scheduling.

To decrease the overheads involved, the users may be encouraged to combine their jobs using an array of jobs. The use of backfilling methods as well as separate queues based on different sizes of jobs is also recommended.

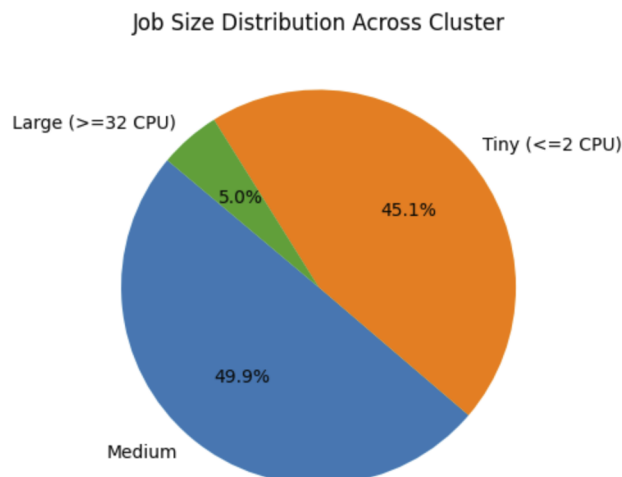


Figure 19: Job Size Distribution Across Cluster

- **New user vs existing user GPU access**

Comparison between new and existing users reveals that existing users use much more GPU than the new ones – not only in terms of total GPU usage, but also when compared by average GPU usage per job. The implication here is that new users encounter difficulties in accessing GPU resources, which could be related to their experience level.

The difference in GPU usage among the two categories of users indicates the presence of obstacles for the former group regarding accessing advanced computing resources. This will lead to delayed user onboarding and lower adoption rates.

Creating onboarding materials, such as tutorials, and sharing sample jobs for new users to run on GPUs will contribute to better GPU utilization by them. Also, setting aside some GPU capacity for the new users would make sense

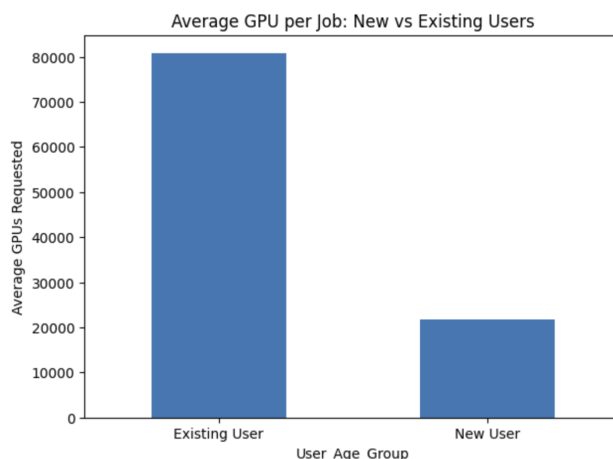


Figure 20: GPU Access: New vs Existing Users

- **Inequality metrics (Gini: user 0.945, GPU 0.998)**

Gini coefficients for measuring resource inequality were computed as follows: Inequality among job submissions by user: 0.945 Inequality among GPUs used: 0.998

The Gini coefficient value being so large in both cases implies that there is too much inequality when distributing resources. Such inequality is detrimental to efficiency and could even lead to a monopoly situation.

The problem of structural inequalities needs to be addressed through the implementation of fairness-aware scheduling policies and continuous monitoring processes. Quota-based allocation schemes which adapt according to past usage trends can also assist in achieving resource balance.

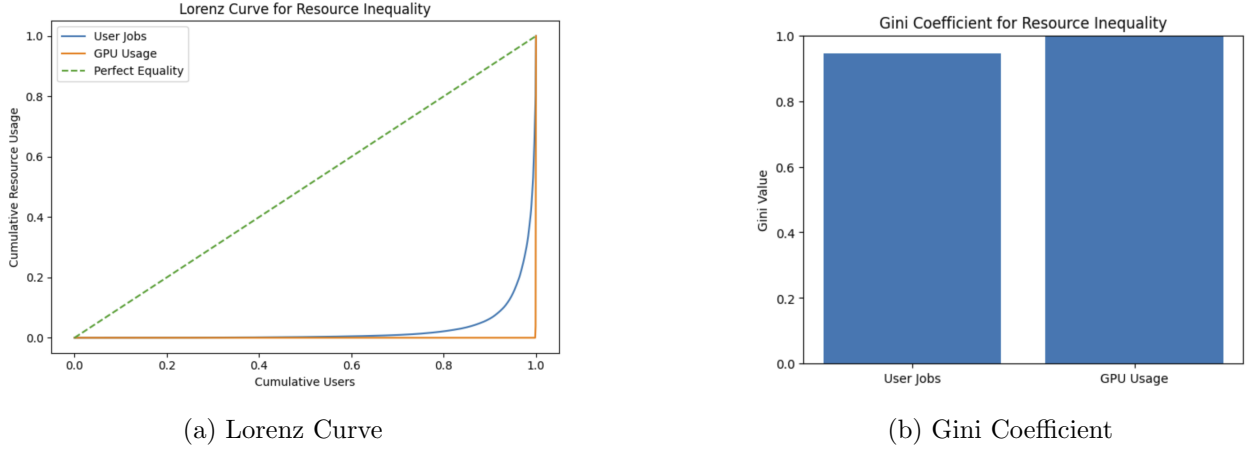


Figure 21: Inequality Metrics: Lorenz Curve and Gini Coefficient

4.3 Sectional Summary

This subsection provided an analysis of the user and resource inequality in the cluster. Several aspects were discussed in order to understand how resource usage and user actions affect cluster efficiency and equality.

First of all, it was noted that the workload distribution is highly polarized across several categories: users, GPUs, departments and sizes of jobs. This means that the vast amount of work is carried out by only some users and/or some departments.

Thus, there is an intensive submission of jobs by a small percentage of users. Similarly, GPU resource usage is highly concentrated among some users, implying the limited access to the powerful resource. Moreover, at the institutional level, the work done by some departments is overwhelming in terms of CPU usage.

Another important aspect is the size of the job, which indicates that there is a large number of very small tasks along with a low number of big ones. As a result, there is high overhead due to the many small jobs, while the large ones lead to blocking and fragmentation.

In conclusion, it can be seen that the cluster is facing structural imbalances in terms of utilization of its resources. It is important that the above-discussed concerns be addressed using fair scheduling techniques and resource allocation policies for effective utilization of the cluster.

5 Contention and Congestion (How)

5.1 Resource Contention

5.1.1 GPU Jobs Experience Longer Waiting Times

To investigate whether GPU jobs exhibit different waiting time behavior, we compare GPU and CPU-only jobs.

Because the waiting time data are non-normal, highly skewed and contain extreme values, a non-parametric statistical test was used to compare the two groups. Specifically, the Mann–Whitney U test was applied to determine whether the distributions of waiting times for CPU-only jobs and GPU jobs differ significantly.

The test returned a p-value of approximately 0 (the Statistic is 1.05×10^{12} , leading to a nearly 0 p-value), which is far below the conventional significance level of 0.05. Therefore, we reject the null hypothesis that the waiting time distributions for CPU and GPU jobs are the same.

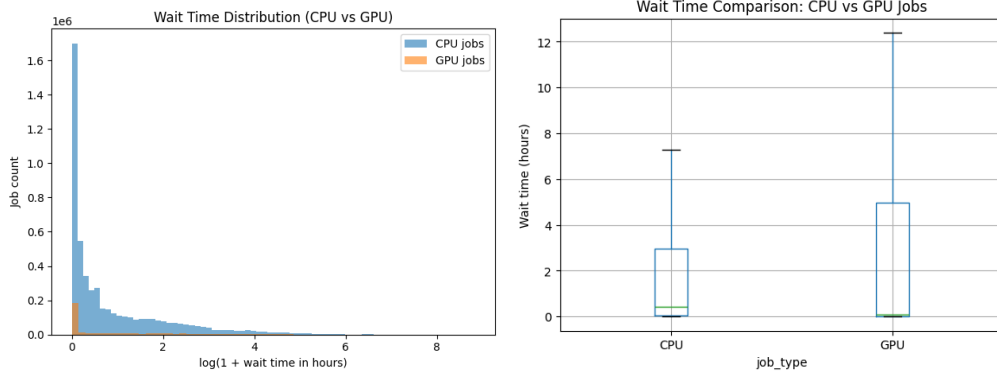


Figure 22: Comparison of waiting times between GPU and CPU-only jobs.

The summary statistics and Figure 22 indicate that GPU jobs tend to experience longer waiting times overall. The mean waiting time for GPU jobs is 8.81 hours, compared with 6.83 hours for CPU-only jobs. However, both distributions are highly skewed with long right tails. For example, the median waiting time for GPU jobs is only 0.087 hours, while the median waiting time for CPU-only jobs is 0.44 hours, indicating that many jobs start quickly but a small number experience very long delays.

A likely explanation is that GPU resources are more limited and subject to higher demand, leading to longer queue times for GPU-based workloads.

5.1.2 Workload-Level Waiting Time Heterogeneity

We further examine waiting time behavior across the constructed workload categories: Light, Medium, and Heavy. Figure 23 compares the median and mean waiting times by workload, while Figure 24 shows the distribution and empirical CDF of waiting times.

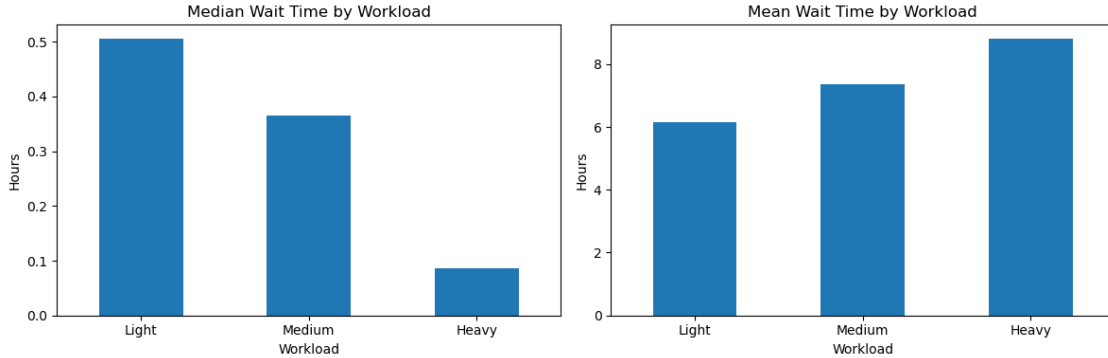


Figure 23: Median and mean waiting time by workload category.

The results show a clear discrepancy between typical waiting time and average waiting time. Heavy workloads have the shortest median waiting time, approximately 0.09 hours, compared with 0.51 hours for Light workloads and 0.37 hours for Medium workloads. This suggests that many Heavy jobs are scheduled relatively quickly, possibly because GPU-intensive jobs are fewer in number and may use more specialized resources.

However, Heavy workloads also have the largest mean waiting time, approximately 8.81 hours, compared with 6.15 hours for Light workloads and 7.37 hours for Medium workloads. This indicates that the Heavy workload distribution has a much stronger right tail: while many Heavy jobs start quickly, a smaller subset experiences substantial delays.

The boxplot and CDF further support this pattern. Heavy workloads exhibit greater variability and a longer upper tail, suggesting more heterogeneous scheduling outcomes. In the CDF plot, Heavy jobs

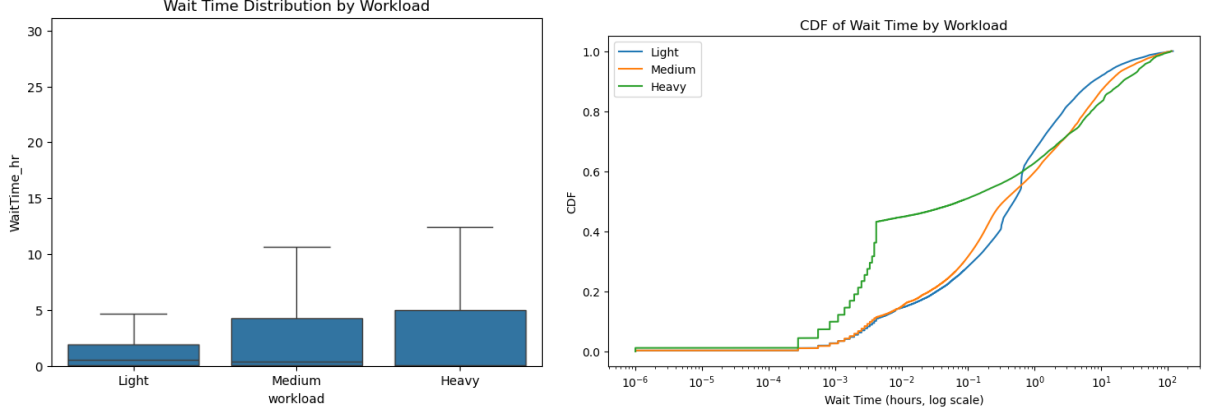


Figure 24: Waiting time distribution and empirical CDF by workload category.

accumulate quickly at very short wait times, but the curve rises more slowly in the upper tail, showing that some Heavy jobs remain in the queue for much longer.

Overall, these results suggest that resource contention is not uniform across workload types. For Heavy workloads, contention appears to be concentrated in the tail: most jobs are scheduled efficiently, but a subset faces severe delays due to competition for scarce GPU or high-memory resources. This two-phase behavior helps explain why Heavy workloads can have both the shortest median waiting time and the largest mean waiting time.

5.1.3 Workload-Level Runtime Differences

We next examine how execution time varies across workload categories. Figure 25 presents the median and mean runtime for each workload group, while Figure 26 shows the distribution of runtime on a log scale.

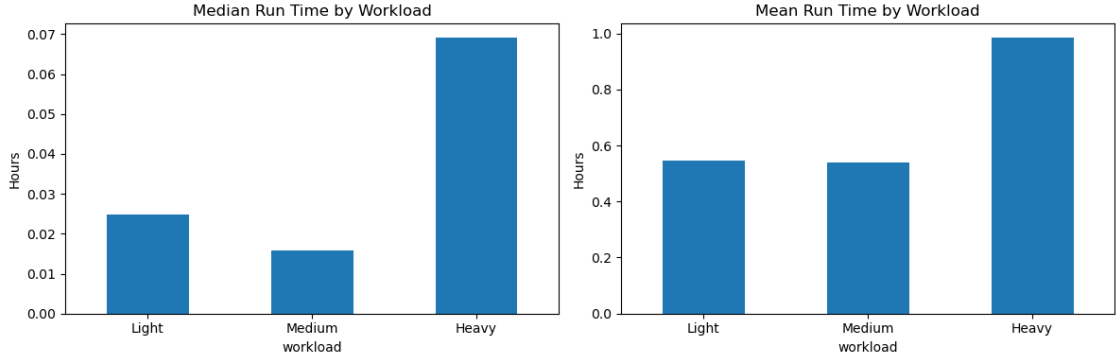


Figure 25: Median and mean runtime by workload category.

We observe that Heavy workloads exhibit longer runtimes, both in terms of median (0.07 hours) and mean (0.99 hours), reflecting their higher computational demands. In contrast, Light and Medium workloads show very similar runtime distributions, with median runtimes around 0.02 hours and mean runtimes around 0.54–0.55 hours, suggesting limited differentiation among CPU-based jobs.

Comparing these results with the waiting time patterns observed earlier reveals an important contrast. Although Heavy workloads take longer to execute, they have shorter median waiting times. This indicates that execution duration alone does not determine scheduling delay.

Instead, waiting time appears to be more strongly influenced by resource availability and contention. In particular, the longer runtime of Heavy jobs implies that once scheduled, these jobs occupy resources for extended periods, reducing system availability for subsequent jobs. This persistence effect contributes to

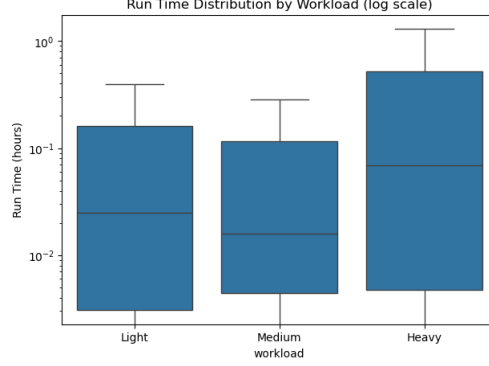


Figure 26: Runtime distribution by workload category (log scale).

resource contention, especially under high load or when specialized resources such as GPUs are limited.

Overall, these findings highlight that workload heterogeneity affects both the duration and availability of resources, reinforcing the role of Heavy jobs in shaping system-level contention dynamics.

5.2 Queue Congestion

5.2.1 Temporal Variation in Waiting Time

We examine how waiting time varies across submission hour and weekday. Figure 27 and Figure ?? summarize the median and mean waiting times.

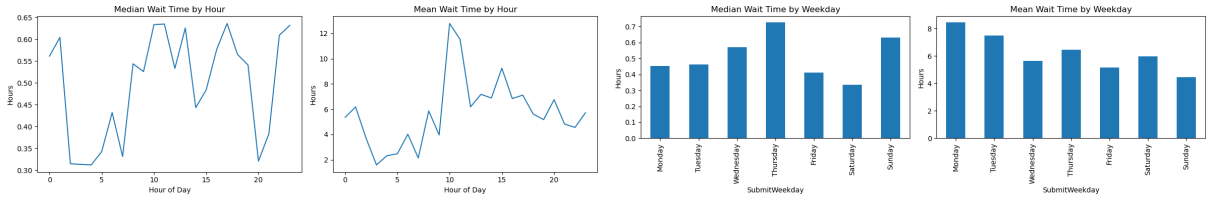


Figure 27: Median and mean waiting time by hour of day and weekday.

The results show moderate variation across both hours of the day and days of the week. In particular, the mean waiting time exhibits noticeable spikes at certain time periods, indicating the presence of extreme delays during peak congestion.

In contrast, the median waiting time remains relatively stable across different submission times. This suggests that most jobs experience similar waiting times regardless of when they are submitted, while only a subset of jobs is affected by severe delays.

Overall, these findings indicate that submission time influences the likelihood of extreme congestion events rather than the typical scheduling outcome. In other words, temporal effects are more pronounced in the tail of the waiting time distribution than in its central tendency.

5.2.2 Relationship Between Runtime and Waiting Time

To further investigate the mechanism of queue congestion, we examine the relationship between job runtime and waiting time. Figure 28 presents a scatter plot on a log-log scale.

We compute the Spearman rank correlation coefficient between runtime and waiting time, obtaining a value of 0.4238 with a p-value close to 0. While the relationship is statistically significant, the magnitude of the correlation is moderate, indicating only a weak-to-moderate monotonic association.

The scatter plot further shows substantial dispersion, with no clear functional relationship between runtime and waiting time. In particular, jobs with similar runtimes can experience widely varying waiting times, and short jobs may still encounter long delays.

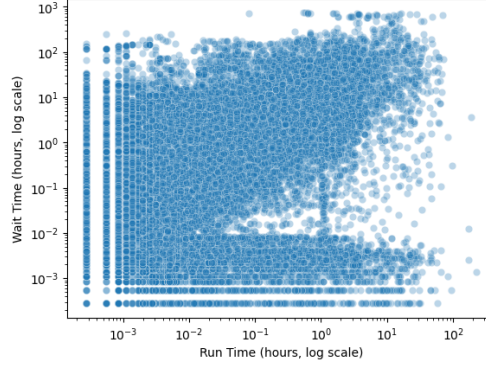


Figure 28: Relationship between runtime and waiting time (log-log scale).

These findings suggest that queue congestion is not determined solely by execution duration. Instead, waiting time is influenced by a combination of factors, including submission timing, workload composition, and resource availability.

Therefore, while long-running jobs contribute to resource occupation, the emergence of queue congestion reflects a more complex system-level interaction rather than a simple one-to-one relationship between runtime and delay.

5.3 Contention and Congestion Sectional Summary

The analyses in this section explain how resource contention and queue congestion emerge from the heterogeneous structure of workloads submitted to the cluster.

First, Heavy jobs have the shortest median waiting time across all workload categories (approximately 0.09 hours), yet also produce the largest mean waiting time (8.81 hours). This divergence arises because a subset of Heavy jobs encounters severe delays driven by competition for scarce GPU and high-memory resources. Once scheduled, these jobs occupy resources for extended periods, further reducing system availability for subsequent jobs. This reveals that contention is a tail phenomenon, not a central-tendency one.

Additionally, submission timing has a measurable but asymmetric effect on waiting time. The median waiting time remains largely stable across hours of the day and days of the week, indicating that most jobs are unaffected by when they are submitted. However, the mean waiting time exhibits pronounced spikes at specific hours, revealing that a minority of submissions encounter extreme delays during peak congestion periods. This pattern implies that congestion events are episodic and burst-driven rather than a consequence of steady background pressure. Temporal effects therefore manifest primarily in the upper tail of the waiting time distribution, not in its typical behavior.

Finally, the Spearman rank correlation between runtime and waiting time is 0.4238 ($p \approx 0$), confirming a statistically significant but only moderate monotonic association. The log-log scatter plot reveals substantial dispersion: jobs with nearly identical runtimes experience widely varying delays, and short jobs are not immune to extreme wait times. This indicates that queue congestion is not reducible to a simple relationship between job length and scheduling delay. Rather, waiting time is shaped by the joint effect of submission timing, workload composition, and instantaneous resource availability, none of which is captured by runtime alone.

- **Takeaway:** Taken together, these findings establish that resource contention and queue congestion are **system-level consequences of workload heterogeneity**, not properties of individual jobs. Heavy jobs create tail-driven contention through resource persistence; peak submission periods generate episodic congestion; and the interaction of workload type, timing, and availability jointly determines which jobs experience extreme delays.
- **Next:** These dynamics set the stage for the job resource inefficiency analysis in the following section.

6 System Inefficiency

6.1 System Inefficiency in Cluster Resource Requests

A major finding from the SLURM job data is that users systematically overestimate the resources required for their jobs. The median time utilization across jobs is only 0.57%, indicating that the typical job uses a very small fraction of the time it requested. In addition, 95.5% of jobs use less than half of their requested runtime.

This may suggest that runtime overestimation is not limited to a small group of extreme cases, but is instead a widespread pattern across the system. However, such small percentage of utilization is also greatly affected by very short jobs, such as jobs that run in less than a minute - the user, in that case, cannot reserve a minute of cluster time, creating a difference of an extreme scale between reserved and requested time. With that said, the median job runtime is approximately 1.2 minutes, while the median requested runtime is 240 minutes. In practical terms, jobs are often around 200 times shorter than the amount of time reserved for them. Across the full dataset, this corresponds to roughly 800 million CPU-hours being reserved but unused.

The degree of overestimation also appears across different job sizes. For example, jobs lasting around five minutes were overestimated by approximately 11.1 times, while jobs lasting around ten minutes were overestimated by approximately 7.5 times. These results show that inefficiency is both large in scale and systematic across the user base.

6.2 Inefficiency Analysis Beyond Runtime

After identifying substantial runtime overestimation, the next question is whether this inefficiency is specific only to requested time or whether it extends to other requested resources. To answer this, we analyzed job-level efficiency records from `seff` data.

Each row in the `seff` dataset corresponds to a completed job and includes both requested and actual resource usage. The key fields used in this analysis include CPU utilization, CPU efficiency, memory usage, and memory efficiency. This allows for a direct comparison between the resources users requested and the resources their jobs actually consumed.

CPU Efficiency

The `seff` data shows that CPU efficiency varies considerably depending on the number of requested cores. Single-core jobs tend to have relatively high efficiency, with a median CPU efficiency of approximately 0.81. However, mid-sized jobs, particularly those requesting between 8 and 32 cores, show very low CPU efficiency.

This pattern suggests that users often over-request CPU resources, especially for mid-sized workloads. While some jobs may require multiple cores, many do not appear to make effective use of the cores allocated to them. As a result, CPU resources may remain reserved but underutilized, reducing overall cluster efficiency.

6.2.1 Memory Efficiency

Memory efficiency shows an even stronger pattern of over-allocation. The median memory efficiency is only 1.48%, with an interquartile range from 0.02% to 15.02%. Additionally, 91.5% of jobs use less than 50% of their requested memory.

These findings indicate that memory over-allocation is widespread and consistent across the cluster. Most jobs request substantially more memory than they ultimately use, which can prevent other jobs from being scheduled even when physical memory remains underutilized.

6.3 Potential Improvements

Several approaches could help reduce resource inefficiency on the cluster. One option is to improve runtime and resource estimation by providing users with historical data guidance. For example, users could be shown summaries of their past job runtimes, CPU usage, and memory usage before submitting new jobs. This would help users make more informed resource requests.

Another possible improvement is the development of lightweight estimation tools. These tools could suggest appropriate runtime, CPU, and memory requests based on prior jobs with similar characteristics. Educational resources may also help users better understand how to estimate job requirements and interpret efficiency metrics.

Finally, flexible runtime extensions could reduce the incentive for users to over-request time. If users are confident that jobs can be extended when necessary, they may be less likely to request excessively long runtimes as a precaution.

However, these solutions depend on both user behavior and system-level changes, meaning adoption and long-term impact are not guaranteed. One concrete approach to encourage user engagement and improve adoption is to provide users with personalized summaries of their cluster usage and resource efficiency metrics.

6.3.1 User Feedback and Personalized Usage Summaries

In addition to identifying inefficiencies in cluster resource allocation, the analysis also motivated a potential user-facing improvement aimed at increasing engagement and encouraging better resource estimation practices.

The core idea is that users are often more engaged when they are given personalized insights into their own behavior and performance. This concept is commonly used in industry applications through yearly recap systems, such as Spotify Wrapped, where users are presented with individualized summaries and comparisons. These summaries encourage users to review, share, and discuss their data in a more interactive way.

Applying a similar concept to cluster usage data, one proposed improvement is to provide each user with a personalized summary of their cluster activity. These summaries could include:

- Runtime efficiency
- CPU efficiency
- Memory efficiency
- Historical trends in resource usage

Additional comparisons could also be included to provide context for users. For example, users could compare their efficiency metrics against overall cluster averages or against similar groups of users with comparable workloads.

This approach has several potential benefits. First, it provides users with direct feedback about how effectively they utilize requested resources. Second, it may encourage users to make more accurate runtime and resource estimates in future job submissions. Finally, presenting efficiency data in a personalized and accessible format may increase overall user engagement with cluster optimization efforts.

From an implementation perspective, this feature is feasible because the project already combines SLURM and `seff` datasets into a unified dataset. As a result, efficiency metrics can be aggregated at the individual user level, enabling the generation of personalized reports without substantial additional infrastructure.

More broadly, this proposal shifts the analysis from being purely descriptive toward a practical improvement strategy. Rather than only identifying inefficiencies, it introduces a user-centered mechanism that could encourage behavioral changes and improve overall cluster utilization over time.

An example of how this may look can be seen in the figures 29, 30 below.

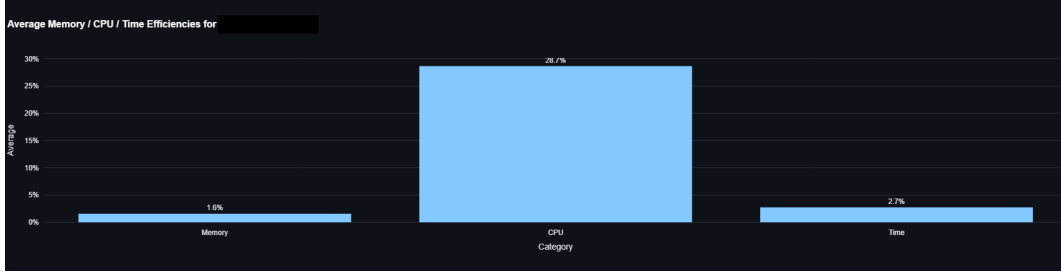


Figure 29: Example personalized cluster usage summary displaying runtime, CPU, and memory efficiency metrics for an individual user.

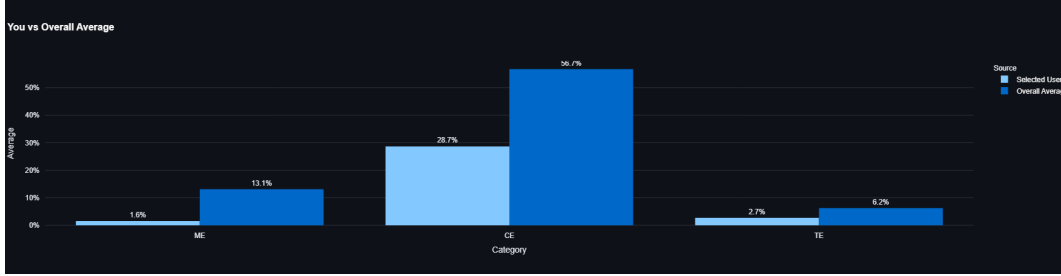


Figure 30: Example comparison of an individual user’s efficiency metrics against overall cluster-wide usage statistics.

6.4 Job Category Efficiency Metric

Individual efficiency metrics (CPU utilization, memory utilization, or time utilization) considered in isolation are insufficient to characterize job-level resource behavior. A job may be highly CPU-efficient while simultaneously wasting the majority of its allocated memory or walltime. To capture this multidimensional structure, we construct a composite efficiency categorization that jointly considers all three dimensions.

Each job is assigned a binary label for each efficiency dimension by thresholding at 0.5: a job is classified as High (H) if its efficiency exceeds 0.5, and Low (L) otherwise. Applying this rule independently to memory efficiency (ME), CPU efficiency (CE), and time efficiency (TE) yields $2^3 = 8$ mutually exclusive and exhaustive efficiency profiles, summarized in Table 1.

Group	Memory	CPU	Time	Interpretation
1	L	L	L	Fully inefficient
2	L	L	H	Only time-efficient
3	L	H	L	Only CPU-efficient
4	L	H	H	CPU + time efficient, memory wasteful
5	H	L	L	Only memory-efficient
6	H	L	H	Memory + time efficient, CPU wasteful
7	H	H	L	Memory + CPU efficient, time wasteful
8	H	H	H	Fully efficient

Table 1: Eight-group efficiency taxonomy based on binary classification of memory, CPU, and time efficiency. H and L denote above and below the 0.5 threshold, respectively.

This taxonomy is designed to capture heterogeneous efficiency *profiles* rather than aggregate performance. Two jobs with identical mean efficiency scores may belong to entirely different groups if their per-dimension breakdowns differ, making this categorization more informative for diagnosing the specific nature of resource waste.

6.4.1 Group Distribution and Job Outcomes

Figure 31 shows the proportion of completed and failed jobs within each efficiency group.

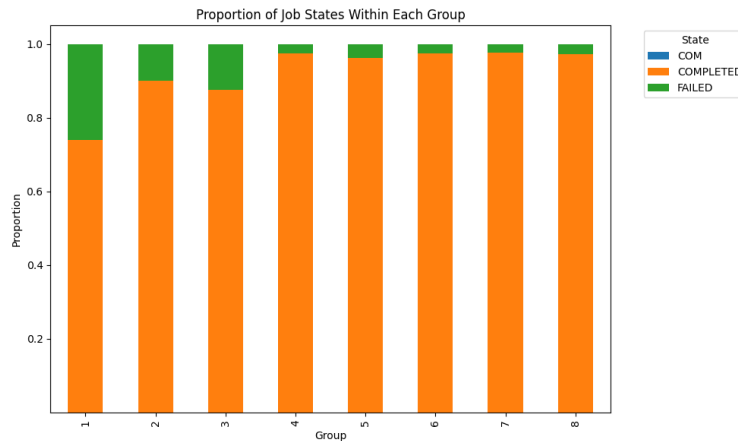


Figure 31: Proportion of job states (COMPLETED vs. FAILED) within each efficiency group.

The distribution of jobs across groups reveals that Groups 3 (L/H/L) and 7 (H/H/L) are by far the most populated, indicating that high CPU efficiency is the most commonly achieved dimension across the cluster. Group 8 - the fully efficient profile - is comparatively rare, suggesting that simultaneously achieving high efficiency across all three dimensions is uncommon in practice. More importantly, the proportion of failed jobs varies substantially across groups, with failure rates visibly elevated in the low-efficiency groups. This pattern implies that job failures are not randomly distributed across efficiency profiles but are instead structurally concentrated among jobs that underutilize their allocated resources.

6.4.2 Statistical Validation: Chi-Squared Test of Independence

To formally test whether failure rates differ across the eight efficiency groups, we apply a chi-squared test of independence to the contingency table of group $\times$ job state (COMPLETED or FAILED). The overall test yields $\chi^2 = 55,557.2$ with $df = 7$ and $p \approx 0$, providing overwhelming evidence that failure rates are not equal across groups.

To identify which specific groups drive this difference, we conduct all 28 pairwise chi-squared tests across the eight groups, applying a Bonferroni correction to control the familywise error rate. The resulting failure counts and rates are reported in Table 2.

Group	Completed	Failed	Failure Rate
1 (L/L/L)	383,719	134,455	26.0%
2 (L/L/H)	9,268	1,017	9.9%
3 (L/H/L)	529,255	74,763	12.4%
4 (L/H/H)	12,600	318	2.5%
5 (H/L/L)	31,936	1,289	3.9%
6 (H/L/H)	5,942	152	2.5%
7 (H/H/L)	59,359	1,491	2.4%
8 (H/H/H)	5,450	151	2.7%

Table 2: Completed and failed job counts by efficiency group, with failure rates.

The pairwise comparisons reveal a clear structural divide. Groups 1, 2, and 3 - all sharing low memory efficiency - have failure rates of 26.0%, 9.9%, and 12.4% respectively, and each differs significantly from the high-memory-efficiency groups under Bonferroni correction. In contrast, Groups 4, 6, 7, and 8 - all characterized by high memory efficiency - cluster tightly around a failure rate of approximately 2.5%, and no

statistically significant difference is found among any pair within this cluster ($p_{\text{Bonf}} = 1.0$ for all six pairings). Notably, CPU efficiency and time efficiency do not independently predict failure: Groups 4 and 7 differ in both CPU and time classification yet are statistically indistinguishable in failure behavior.

These results establish that **memory efficiency** is a critical factor in job failures. Once a job achieves high memory efficiency, its failure rate drops sharply and stabilizes regardless of its CPU or time efficiency profile. Low memory efficiency, by contrast, is associated with substantially elevated failure risk across all combinations of the other two dimensions.

6.4.3 Within-Group Memory Efficiency Distributions

To validate that the binary threshold creates meaningful separation, Figure 32 presents boxplots of memory efficiency by group and job state.

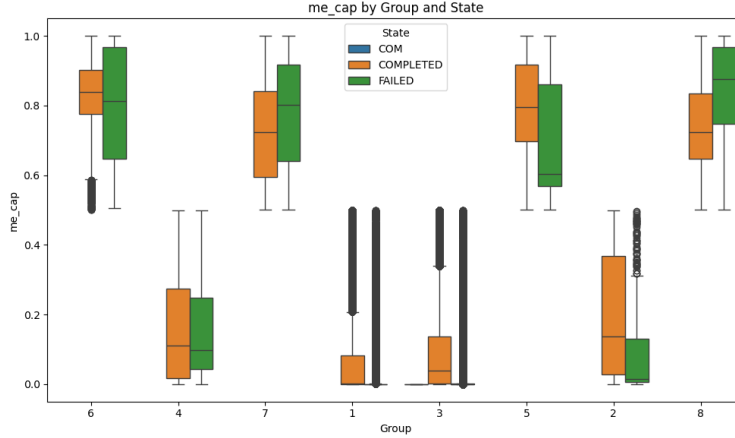


Figure 32: Memory efficiency distributions by efficiency group and job state.

The plot confirms that the 0.5 threshold produces clean and meaningful separation. High-memory groups (5, 6, 7, 8) concentrate tightly in the $[0.6, 1.0]$ range, while low-memory groups (1, 2, 3, 4) collapse near zero with limited spread. An additional finding concerns failed jobs within high-memory groups: these jobs retain high memory efficiency despite their failure, indicating that memory waste is not the cause of failure in those cases. Instead, failure is attributable to other bottlenecks, such as CPU contention, walltime exhaustion, or application-level errors, that are not captured by the memory dimension alone. This confirms that this job category efficiency metric identifies structural differences in resource behavior.

6.5 Sectional Summary

This section identified substantial inefficiencies in cluster resource allocation across runtime, CPU, and memory usage. Runtime requests were frequently overestimated, with most jobs using only a small fraction of their reserved walltime. Similar patterns were observed for CPU and memory allocations, indicating that resource over-requesting is a widespread and systematic issue rather than an isolated behavior.

To better characterize these inefficiencies, a multidimensional efficiency taxonomy was introduced that jointly classified jobs based on memory, CPU, and time efficiency. The resulting analysis showed that low memory efficiency is strongly associated with elevated job failure rates, while high memory efficiency corresponds to substantially lower and more stable failure behavior across groups. These findings suggest that memory allocation behavior may be one of the most important indicators of overall job reliability.

Finally, several potential improvement strategies were discussed, including historical usage guidance, lightweight estimation tools, flexible runtime extensions, and personalized usage summaries. Together, these approaches aim not only to improve cluster utilization, but also to encourage more informed and efficient user behavior over time.

7 Predictive Modeling

7.1 Predicting GPU Job Waiting Time

Based on the results in Section 5.1.1, we construct separate prediction models for CPU-only and GPU jobs, as these two types exhibit different distributions. We think that different distributions suggest different underlying mechanisms, so we build models for these two types of jobs separately.

We start with building prediction model for GPU-job waiting times.

In this part, we implemented a direct regression approach to predict GPU job waiting time. The response variable was defined as the waiting time in hours:

$$\text{WaitTimeHr} = \frac{\text{WaitTimeSec}}{3600}.$$

Before model fitting, we cleaned the data by removing observations with missing, infinite, or negative waiting times. Then we extracted several scheduling-related features, including submission weekday, submission hour, QOS, GPU count, requested CPUs, requested memory, and job priority. They can be divided into two types:

- Categorical: ‘SubmitWeekday’, ‘SubmitHour’, ‘QOS’;
- Numeric: ‘GPU_Count’, ‘ReqCPUS’, ‘ReqMem_MB’, ‘Priority’.

Categorical variables were encoded using one-hot encoding, while numerical variables were standardized using z-score scaling.

To reduce the influence of extreme numerical values, we applied winsorization to the numerical features using the 2.5% and 97.5% quantiles computed from the training data. Missing numerical variables were imputed by the median and standardized, while categorical variables were imputed by the most frequent value and encoded using one-hot encoding. We randomly split the data into 80% training and 20% testing sets using a fixed random seed to ensure reproducibility.

Because the waiting-time distribution is highly right-skewed, both models were trained on the log-transformed response:

$$\log(1 + \text{WaitTimeHr}).$$

After prediction, the fitted values were transformed back to the original hour scale using the inverse transformation:

$$\widehat{\text{WaitTimeHr}} = \exp(\hat{y}_{\log}) - 1.$$

We compared two nonlinear regression models: XGBoost and Random Forest. Both models used the same preprocessing pipeline and the same train-test split, so their performance could be compared fairly.

Model	MAE	RMSE	RMSLE
XGBoost	2.5583 hours	24.4489 hours	0.5377
Random Forest	1.9979 hours	21.9759 hours	0.5104

Table 3: Prediction performance of direct regression models without grouping.

Based on these results, the Random Forest model performs slightly better than XGBoost under this situation. It has lower MAE, RMSE, and RMSLE values, indicating that it produces more accurate predictions on average and also performs better on the log scale.

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}.$$

The RMSLE metric is especially useful in this problem because the waiting-time distribution is highly skewed. Compared with RMSE, RMSLE gives more emphasis to relative prediction errors and is less dominated by extremely large waiting times.

The results in Table 3 suggest that our predictions deviated from the true waiting times by approximately 2 hours on average, and were typically within a multiplicative factor of 1.67 of the true values.

The log-scale prediction plot Figure 33 compares the predicted waiting times with the true waiting times. The red dashed line represents the ideal case where the predicted value equals the true value. Points close to this line indicate accurate predictions. From the plot, both models capture the general increasing trend for many observations, especially in the moderate waiting-time range. However, there are still noticeable deviations for very short and very long waiting times, suggesting that predicting extreme waiting times remains difficult.

In addition, the plot shows several horizontal patterns in the predicted values. This is reasonable because both Random Forest and XGBoost are tree-based models, and their predictions are formed from piecewise-constant partitions of the feature space. As a result, multiple observations may receive similar or identical predicted waiting times.

Overall, the direct regression approach provides a simple baseline for wait-time prediction without using a prior classification step. Among the two models, Random Forest gives better predictive performance in this experiment. However, the RMSE is much larger than the MAE for both models, which suggests that a small number of extreme waiting-time observations still lead to large prediction errors. Therefore, it may be useful to compare this direct regression approach with the two-stage classification-plus-regression approach, especially for handling highly imbalanced waiting-time ranges.

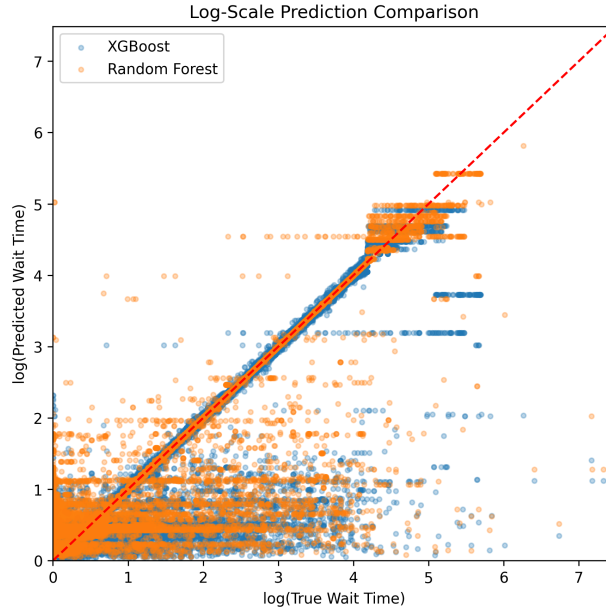


Figure 33: Log-Scale Prediction Comparison (GPU)

7.2 Predicting CPU-only Job Waiting Time

In addition to GPU jobs, we also implemented a prediction model for CPU-only jobs. In this analysis, CPU-only jobs were defined as jobs with

$$\text{GPU_Count} = 0.$$

After filtering the data to CPU-only jobs, we applied the same data cleaning and preprocessing procedure as in the GPU-job prediction. In CPU-only job prediction, we use the same set of features as in the GPU job prediction. We use the same methods to deal with missing values, sampling and wisorization.

Because wait times for CPU-only jobs are highly right-skewed, we trained the Random Forest model using the log-transformed response:

$$y_{\log} = \log(1 + \text{WaitTimeHr}).$$

After prediction, the fitted values were transformed back to the original hour scale by

$$\widehat{\text{WaitTimeHr}} = \exp(\hat{y}_{\log}) - 1.$$

The Random Forest model was fitted with 200 trees and maximum tree depth 10. The model performance on the test set is shown below:

Metric	Value
MAE	3.9319 hours
RMSE	23.3144 hours
RMSLE	0.6559

Table 4: Prediction performance of the Random Forest model for CPU-only jobs.

The results in Table 4 suggest that our predictions deviated from the true waiting times by approximately 3.9 hours on average, and were typically within a multiplicative factor of 1.9 of the true values.

Compared with the GPU-job prediction results, the CPU-only model has a larger MAE and RMSLE. This suggests that CPU-only job waiting times are harder to predict accurately using the current set of features, and the distribution of CPU-only jobs' waiting time is more complicated. Although the RMSE is still large, it is close to the GPU-job RMSE, which indicates that both CPU and GPU wait-time prediction are affected by a small number of extreme waiting-time observations.

The log-scale scatter plot compares the predicted waiting times with the true waiting times for CPU-only jobs. The red dashed line represents perfect prediction. Points close to this line indicate accurate predictions. From the plot, the model captures some general increasing trend, but the points are widely dispersed around the diagonal line. There are also several horizontal bands in the predicted values, which is expected for a tree-based model because Random Forest predictions are piecewise constant within terminal nodes.

The plot also shows that the model tends to overpredict some very short waiting times and underpredict some very long waiting times, while perform well in mid-range waiting time prediction. This pattern indicates that the model has difficulty capturing extreme queueing behavior. Overall, the Random Forest model provides a useful baseline for CPU-only wait-time prediction, but the relatively high MAE and RMSLE suggest that additional features or a more refined modeling strategy may be needed to improve prediction accuracy for CPU jobs.

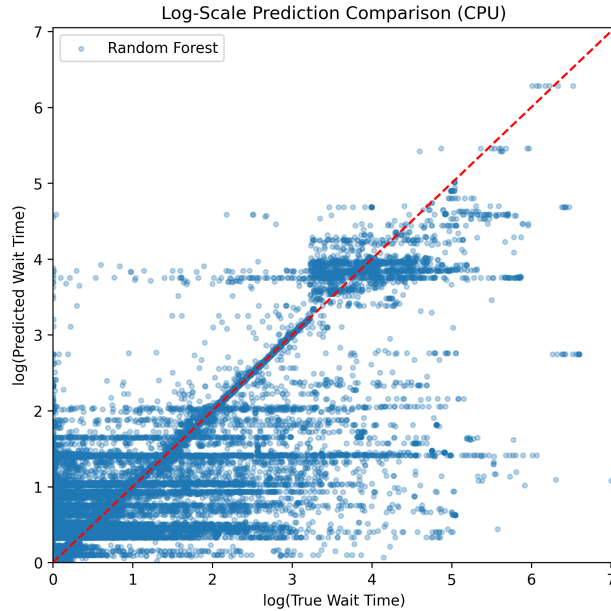


Figure 34: Log-Scale Prediction Comparison (CPU)

7.3 Predicting Heavy Users and System-Level Behavior

In addition to queue wait-time prediction, our study also explored user-level behavioral modeling to identify dominant users whose workloads significantly influence overall cluster utilization. Heavy users were defined as users whose resource-consumption characteristics — including number of submitted jobs, CPU usage, GPU usage, runtime, and departmental diversity — place them within the top 10% of all users in the system.

To classify whether a user belongs to the heavy-user category, a Random Forest classification model was developed and evaluated. The model achieved an accuracy of approximately 85% along with a ROC-AUC score of 0.82, indicating strong predictive capability and reliable classification performance.

Feature-importance analysis revealed that average CPU utilization, total GPU consumption, and average runtime are among the most influential factors in identifying heavy users. These findings demonstrate that user behavior within HPC environments exhibits predictable patterns that can be effectively modeled using machine learning techniques.

Such predictive insights are valuable for proactive resource-management strategies. By identifying dominant users early, system administrators can reduce the risk of resource monopolization, improve workload balancing, and support fairer scheduling decisions across departments.

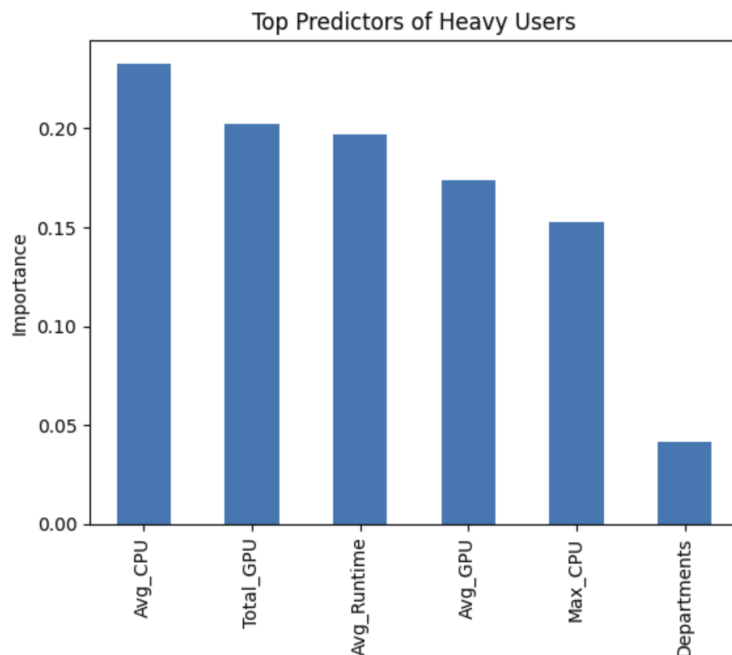


Figure 35: Feature importance scores for predicting heavy users in the HPC cluster system.

7.4 Sectional Summary

- The heavy-user prediction model successfully identifies dominant users with high classification accuracy.
- Resource-consumption metrics such as CPU usage, GPU utilization, and runtime are strong indicators of user behavior.
- Predictive modeling enables proactive identification of potential resource monopolization within the cluster.
- Integrating user-behavior predictions with scheduling policies can improve fairness, efficiency, and overall cluster performance.

8 Conclusion and Recommendations

8.1 Key Findings and System Implications

- While the cluster system runs effectively under normal workloads, it exhibits poor performance and imbalance when confronted with either extreme or high demands. A minor percentage of users makes up the bulk of job submissions and GPU utilization, leading to highly skewed workloads and monopolistic use of resources.
- Other insights gained indicate that GPUs have been used mostly by few users, while CPUs have been utilized mainly by a few departments. Moreover, there exists a high presence of small jobs together with huge jobs in terms of resource utilization in the workload distribution, which has negative implications on efficiency.
- From the predictive modeling, it is evident that heavy users can be predicted through workload and resource features like CPU allocation, GPU utilization, and runtimes. The findings indicate that user behavior has a vital role to play in determining congestion and fairness.
- We also find that resource waste is widespread and multidimensional - median memory efficiency is 1.48%; ~800 million CPU-hours reserved but unused. Inefficiency is not limited to one resource - our 8-group job category metric of efficiency reveals that most jobs are efficient in at most one or two dimensions.
- The same job category metric also reveals that job failures are not random - they are structurally linked to efficiency profiles. Failed jobs concentrate in low-efficiency groups. Jobs that fail within high-memory groups do so due to other bottlenecks, confirming that the efficiency taxonomy captures meaningful behavior.

8.2 Recommendations to the Client

Based on these findings, we propose the following actionable improvements:

1. Monitor and manage heavy users

A small fraction of users contributes disproportionately to job submissions and resource demand. Fair-share policies, soft quotas, or usage-based priority adjustments could help reduce imbalance while preserving access for high-throughput users.

2. Use predictive models as decision support

Our predictive models provide a baseline for estimating waiting times from information available at submission time. Rather than replacing the scheduler, these models could support operational decisions by flagging jobs that may be delay-prone or high-impact.

3. Enhance predictive models with richer contextual features

Current predictive models rely primarily on coarse job-level features, which limits their ability to capture long-tail waiting-time behavior. Incorporating additional contextual information, such as recent queue load, historical user behavior, department-level usage patterns, or job type, could improve prediction accuracy in a privacy-preserving and system-compatible manner.

4. Introduce fairness-aware resource allocation strategies

The analysis reveals substantial inequality in resource distribution across users and departments, particularly in GPU usage. Implementing fairness-aware scheduling mechanisms, adaptive quotas, and department-level balancing policies could help reduce monopolization behavior and improve accessibility for smaller research groups and new users.

5. Personalized Efficiency Feedback

Report per-user efficiency summaries (runtime, CPU, memory) compared against the overall cluster, increasing awareness and incentivizing more accurate resource requests.

6. Targeted Intervention by Efficiency Group

Use the 8-group job category efficiency metric to identify systematically wasteful job profiles and deliver group-specific guidance, prioritizing users whose jobs consistently fall in Groups 1–3.

8.3 Limitations & Future Work

Limitations

1. **Observational nature of the data:** The analysis is based on historical log data, which limits our ability to establish causal relationships between workload characteristics, user behavior, and system outcomes.
2. **Limited feature space:** The available logs primarily capture resource requests and system states, but lack richer contextual information (e.g., job purpose, user intent, or detailed workflow structure), which may constrain the explanatory and predictive power of our models.
3. **Simplified workload representation:** The clustering-based workload variable provides an interpretable summary of resource intensity, but may not capture more complex or dynamic workload patterns.
4. **Model limitations:** The predictive models rely on relatively coarse features and simple structures, which may limit their performance, especially in capturing extreme or long-tail behaviors.
5. **Limited visibility into user intent and workflow behavior:**

Although there appear to be significant trends toward resource clustering and monopoly within the analysis, there is no evidence from the logs regarding the intentions, research goals, or process interdependencies of the users. Therefore, some of the inequalities identified may be the result of efficient computation, not inefficiency or unfairness.

Future Work

1. **Enhance predictive modeling:** Develop more accurate and adaptive models by incorporating richer contextual features, such as historical user behavior and workload patterns.
2. **Incorporate user feedback mechanisms:** Design systems that allow users to iteratively refine runtime estimates, reducing systematic overestimation and improving resource utilization.
3. **Deeper analysis of scheduling dynamics:** Investigate how scheduling policies and priority rules interact with workload heterogeneity to drive contention and congestion.
4. **Broader and longer-term evaluation:** Extend the analysis across different time periods, workloads, and system configurations to assess the robustness and generalizability of the findings.
5. **Develop fairness-aware and adaptive scheduling frameworks:**

Future work may look into incorporating predictive models within scheduling strategies that can preemptively identify heavy users and dynamically allocate resources to accommodate this. This would result in improved fairness, better load balancing, and efficient use of GPUs.

Appendix A: Code and File Structure

The project folder, accessible at this [Box Folder link](#), is organized into three analysis subfolders, a shared preprocessing script, and the raw data file. Each subfolder contains a .ipynb notebook with all code and a PDF or text read-me describing how to run it. The structure is as follows:

```
STAT427-Project/
|
|-- whole_cluster_usage_merged_raw.txt.gz      # Raw merged SLURM logs (full year)
|-- Date Preprocessing Script.ipynb            # Preprocessing pipeline
|-- Date Preprocessing Script Description.pdf    # Instructions for preprocessing
|
|-- Job Submission Behavior & Execution Schedules (Jiachen & Yanlin)/
|   |-- Code Instruction_Job Submission Behavior+Execution Sc... [PDF] # Documentation for this folder
|   |-- master_2025_joblevel_submission.csv.gz  # Job-level submission view
|   |-- master_2025_joblevel_terminal.csv.gz    # Job-level terminal/outcome view
|   |-- master_2025_raw_valid.csv.gz           # Cleaned row-level dataset
|   |-- Job Submission Behavior & Execution Schedules & Queue Modeling.ipynb
|   # ^^ All Analysis and Dataset Construction Code
|
|-- DemoGraphic and Detrimental usage Analysis (Rushil)/
|   |-- whole_data_removed_columns (1).csv      # Input dataset for user analysis
|   '--- 427_Project_Rushil (1).ipynb          # User behavior & inequality analysis
|
'--- System Inefficiency Analyses and App Demo (Vlad & Rishab)/
    |-- cluster_efficiency_dataset.csv          # Merged SLURM + seff efficiency data
    |-- cluster_efficiency_groups_only.csv      # Job category group assignments
    |-- Seff/                                  # Raw seff data files
    |-- Intermediate Data/                     # Intermediate processing outputs
    |-- Analysis Pipeline/                     # Inefficiency analysis notebooks
    '--- User Feedback/                        # Personalized feedback app files
```

Navigation Guide

Step 1: Data Preprocessing. Begin with `Date Preprocessing Script.ipynb`, which ingests `whole_cluster_usage_merged_raw.txt.gz` and produces the three job-level datasets used throughout the project. The accompanying PDF describes all preprocessing steps and output schemas in detail.

Step 2: System Baseline, Contention, and Congestion Analysis. Open the notebook in `Job Submission Behavior & Execution Schedules (Jiachen & Yanlin)/`. This folder contains the three processed datasets and a PDF code instruction file. The notebook covers all analyses in Sections 3 and 5 of the report, including submission patterns, waiting time distributions, workload clustering, contention, and queue congestion.

Step 3: User Behavior and Resource Inequality Analysis. Open `427_Project_Rushil (1).ipynb` in `DemoGraphic and Detrimental usage Analysis (Rushil)/`. This notebook covers all analyses in Section 4, including heavy user concentration, GPU and CPU inequality, department-level usage, job size distribution, and Gini coefficient computation.

Step 4: System Inefficiency and Job Category Analysis. Navigate to `System Inefficiency Analyses and App Demo (Vlad & Rishab)/`. The `Analysis Pipeline/` subfolder contains notebooks for the efficiency analyses in Section 6, including runtime overestimation, memory and CPU efficiency, the eight-group efficiency taxonomy. The `User Feedback/` subfolder contains the personalized usage summary application. The two root-level CSVs (`cluster_efficiency_dataset.csv` and `cluster_efficiency_groups_only.csv`) are the primary inputs for these analyses and are produced by merging the SLURM terminal view with the `Seff/` data.

Note: All notebooks are self-contained and include inline comments. Each analysis subfolder's read-me (PDF or text) provides additional detail on dependencies, runtime, and expected outputs.